

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”

Курс «Парадигмы и конструкции языков программирования»

Отчет по лабораторной работе №2

«Объектно-ориентированные возможности языка Python»

Выполнил:
студент группы ИУ5 - 32Б:
Акопов А.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Надрид А. Н.
Подпись и дата:

Москва, 2025 г.

Задание:

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием `pip`.
2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке Python 3.
3. Все файлы проекта (кроме основного файла `main.py`) должны располагаться в пакете `lab_python_oop`.
4. Каждый из нижеперечисленных классов должен располагаться в отдельном файле пакета `lab_python_oop`.
5. Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для вычисления площади фигуры. Подробнее про абстрактные классы и методы Вы можете прочитать [здесь](#).
6. Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры. Подробнее про описание свойств Вы можете прочитать [здесь](#).
7. Класс «Прямоугольник» наследуется от класса «Геометрическая фигура». Класс должен содержать конструктор по параметрам «ширина», «высота» и «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий площадь фигуры.
8. Класс «Круг» создается аналогично классу «Прямоугольник», задается параметр «радиус». Для вычисления площади используется константа `math.pi` из модуля [math](#).
9. Класс «Квадрат» наследуется от класса «Прямоугольник». Класс должен содержать конструктор по длине стороны. Для классов «Прямоугольник», «Квадрат», «Круг»:
 - Определите метод `getр`, который возвращает в виде строки основные параметры фигуры, ее цвет и площадь. Используйте метод `format` - <https://pyformat.info/>
 - Название фигуры («Прямоугольник», «Квадрат», «Круг») должно задаваться в виде поля данных класса и возвращаться методом класса.
10. В корневом каталоге проекта создайте файл `main.py` для тестирования Ваших классов (используйте следующую конструкцию - https://docs.python.org/3/library/_main_.html). Создайте следующие объекты и выведите о них информацию в консоль (N - номер Вашего варианта по списку группы):
 - Прямоугольник синего цвета шириной N и высотой N.
 - Круг зеленого цвета радиусом N.
 - Квадрат красного цвета со стороной N.
 - Также вызовите один из методов внешнего пакета, установленного с использованием `pip`.

11. **Дополнительное задание.** Протестируйте корректность работы Вашей программы с помощью модульного теста.

Код:

Папка lab_python_oop:

shape.py

```
from abc import ABC, abstractmethod

class Figure(ABC):
    @abstractmethod
    def area(self):
        pass
```

shape_color.py

```
class Color:
    def __init__(self):
        self._color = None

    @property
    def colorproperty(self):
        return self._color

    @colorproperty.setter
    def colorproperty(self, value):
        self._color = value
```

circle.py

```
from lab_python_oop.shape import Figure
from lab_python_oop.shape_color import Color
import math

class Circle(Figure):
    FIGURE_TYPE = "Круг"

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE
```

```

def __init__(self, color_param, r_param):
    self.r = r_param
    self.fc = Color()
    self.fc.colorproperty = color_param

def area(self):
    return math.pi*(self.r**2)

def __repr__(self):
    return '{} {} цвета радиусом {} площадью {}'.format(
        Circle.get_figure_type(),
        self.fc.colorproperty,
        self.r,
        self.area()
    )

```

rectangle.py

```

from lab_python_oop.shape import Figure
from lab_python_oop.shape_color import Color

class Rectangle(Figure):
    FIGURE_TYPE = "Прямоугольник"

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, color_param, width_param, height_param):
        self.width = width_param
        self.height = height_param
        self.fc = Color()
        self.fc.colorproperty = color_param

    def area(self):
        return self.width*self.height

    def __repr__(self):
        return '{} {} цвета шириной {} и высотой {} площадью {}'.format(
            Rectangle.get_figure_type(),
            self.fc.colorproperty,
            self.width,
            self.height,
            self.area()
        )

```

square.py

```
from lab_python_oop.rectangle import Rectangle

class Square(Rectangle):
    """
    Класс «Квадрат» наследуется от класса «Прямоугольник».
    """
    FIGURE_TYPE = "Квадрат"

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, color_param, side_param):
        """
        Класс должен содержать конструктор по параметрам «сторона» и «цвет».
        """
        self.side = side_param
        super().__init__(color_param, self.side, self.side)

    def __repr__(self):
        return '{} {} цвета со стороной {} площадью {}.'.format(
            Square.get_figure_type(),
            self.fc.colorproperty,
            self.side,
            self.area()
        )
```

папка lab2:

main.py

```
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

from colorama import Fore
def main():
    # Параметры фигур (вариант 2, N = 2)
    N = 2

    # Создание фигур
    rectangle = Rectangle("белого", N, N)
    circle = Circle("синего", N)
```

```

square = Square("красного", N)

# Вывод информации о фигурах
print(Fore.WHITE + str(rectangle))
print(Fore.BLUE + str(circle))
print(Fore.RED + str(square))

if __name__ == "__main__":
    main()

```

test_colors.py

```

import unittest
import math
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

class TestArea(unittest.TestCase):

    def test_rectangle_area(self):
        rect = Rectangle("Синий", 2, 2)
        self.assertEqual(rect.area(), 4.0)

    def test_circle_area(self):
        rect = Circle("Зеленый", 2)
        self.assertEqual(rect.area(), math.pi * 2 ** 2)

    def test_square_area(self):
        rect = Square("Красный", 2)
        self.assertEqual(rect.area(), 4.0)

if __name__ == '__main__':
    unittest.main()

```

Скриншот работы:

```

(venv) (base) an.akopov@MacBook-Air-Akopov lab2 % python3 main.py
Прямоугольник белого цвета шириной 2 и высотой 2 площадью 4.
Круг синего цвета радиусом 2 площадью 12.566370614359172.
Квадрат красного цвета со стороной 2 площадью 4.
(venv) (base) an.akopov@MacBook-Air-Akopov lab2 % python3 test_colors.py
...
-----
Ran 3 tests in 0.000s

OK

```